

GatePass

Event parking marketplace — project summary & schema | Draft v2

"GatePass" is a working name only — trademark (IP India), domain, and app-store name checks are still pending before it's final.

1. What this project is

A paid, two-sided marketplace for parking at **public ticketed events** in India — concerts, sports matches, exhibitions, fairs. Event organizers list available parking capacity at their venue; drivers discover and pay to reserve a spot in advance; the platform issues a digital QR pass for gate entry.

Why this segment, and not the others considered

Segment	Verdict	Reason
General P2P (any driveway, any city)	Rejected	Most Indian urban parking sits in apartment societies. Supreme Court (Nahalchand, 2010) and RERA treat allotted parking as a common area controlled by the RWA, not the flat owner's private property to rent out. Legally clean supply is much smaller than the UK/US model this idea was based on.
Private weddings / banquet halls	Rejected	A player (MetroPark+) already runs WhatsApp-based digital parking passes for this segment. More fundamentally, charging wedding guests to park is culturally unusual in India — the revenue model (commission on driver payment) doesn't fit; the incumbent monetizes as a B2B logistics tool sold to the venue, not a driver-paid marketplace.
Public ticketed events	Selected	Driver-paid parking is already a normal, accepted behaviour here (stadium parking, event parking). Supply is concentrated (one venue deal covers many spots). No dedicated India-specific competitor found for this exact niche at the time of research.

Builder context

Solo builder, bootstrapped (own time and limited budget), targeting the Indian market. Native mobile app chosen deliberately over a web-first MVP for credibility/portfolio reasons, accepting the extra build time and app-store overhead that comes with it (see Section 3).

2. Business model

Modelled on how Zomato/Swiggy settle with restaurants, not on a pure classifieds (OLX/Quikr) model, and not on a real-time payment-split marketplace (Razorpay Route) model:

- Driver pays the **full amount into the platform's own account** via a standard Razorpay Payment Gateway.
- Platform deducts its **commission** (kept conservative — 10–15% to start; Zomato/Swiggy's 25–35% effective take rate has driven restaurants to complain to the CCI and adopt zero-commission alternatives, which is a cautionary data point, not a benchmark to copy).
- Organizer is paid out on a **periodic settlement** (weekly, or per-event) with an itemized, per-booking breakdown — not a real-time split. This avoids the settlement-reconciliation disputes that Zomato/Swiggy restaurant partners commonly report.

Open item: whether this service category has a specific GST treatment (similar to Section 9(5) CGST for restaurant aggregators) has not been confirmed and needs a CA's input before launch. Business registration route: sole proprietorship + GST is expected to be sufficient for Razorpay KYC at this stage — full Pvt Ltd incorporation is not assumed to be necessary yet.

3. Tech stack

Layer	Choice	Why
Driver app	React Native (Expo, managed workflow)	Native chosen for credibility over web-first MVP. Expo over bare RN CLI for solo dev: handles build/app-store submission (EAS Build/Submit) and OTA JS updates without native reconfiguration.
Organizer dashboard	React (web)	Organizer works from a laptop to create listings and view settlement reports — no app-store distribution needed. Reuses React knowledge from the mobile app.
Backend	Node.js + Fastify + TypeScript	2–3x Express's throughput on JSON-heavy endpoints, built-in JSON Schema (Ajv) request validation, native TypeScript support. TypeScript chosen so booking/payment types can be shared across backend and both frontends, reducing integration bugs on amount/quantity fields.
DB access layer	Drizzle ORM	The oversell-prevention query (Section 4) needs a precise conditional UPDATE. Drizzle stays close to raw SQL with full TypeScript typing; a heavier ORM like Prisma would push that query into an escape-hatch raw-query call, losing most of its own benefit.
Database	PostgreSQL	Supports the atomic capacity-check pattern natively; no exotic extensions needed.
Payments	Razorpay Payment Gateway (not Route)	Full payment collected into the platform's account; payouts to organizers handled as periodic settlements (Section 2), not a real-time split.

Known gotcha: `react-native-razorpay` is a native module and will not run inside Expo Go. Payment testing needs an Expo Dev Client or an EAS development build — a common first surprise for Expo + Razorpay setups.

4. MVP architecture

- **Single monolith service** + Postgres. No microservices or Kubernetes at this stage — there is no scaling problem yet to justify the operational overhead.
- **Oversell prevention** is a single atomic conditional update on a capacity counter (`UPDATE parking_capacity SET booked_count = booked_count + qty WHERE booked_count + qty <= total_capacity`) rather than manual row locking.
- **Idempotency keys** on booking creation so retries from flaky mobile networks never create duplicate bookings.
- **Digital QR pass**: generated server-side on booking confirmation, delivered via WhatsApp/SMS and shown in-app; the venue's existing security staff verifies it visually — no scanner hardware or LPR investment for the MVP.
- **Push notifications deferred**: bookings are one-off per event, not daily-engagement usage, so WhatsApp/SMS confirmation covers it without a Firebase Cloud Messaging setup yet.

Native app adds its own overhead — budget for it

- Apple Developer Program: \$99/year. Google Play: \$25 one-time.
- App Store review cycle (roughly 1–3 days, first submissions can be rejected) — build launch-date buffer around this.
- A CI/CD pipeline to trigger EAS Build/Submit (e.g. from GitHub Actions) is worth setting up early — also a good practitioner-level DevOps artifact for a portfolio.

Deferred until an actual problem forces it

Multi-party payment splitting (Razorpay Route) • geo-search across listings • horizontal scaling • real-time websocket availability • formal fraud/dispute tooling • push notifications (FCM) • native QR-scanning hardware/LPR.

5. Database schema

Renamed since v1: `events` → **listings**, with a new `listing_type` column (only 'event' is used today). This is so a future non-event supply type — e.g. an office building renting out spare parking on a recurring schedule — doesn't require a rename/migration later. The recurring case itself is not built yet: it would be modelled as auto-generated daily capacity rows so the same atomic counter logic keeps working, with no new concurrency design needed.

users

Column	Type	Notes
id	uuid, PK	
phone	varchar, unique, not null	Primary auth identifier
name	varchar	
role	enum(driver, organizer, admin)	
gst_number	varchar, nullable	Organizer only
bank_account_id	varchar, nullable	Razorpay Fund Account ref, for payouts
created at / updated at	timestamp	

Open design question: `gst_number` and `bank_account_id` are always null for drivers. Fine for MVP; consider splitting into a separate `organizer_profiles` table if organizer-specific fields grow.

listings (renamed from events)

Column	Type	Notes
id	uuid, PK	
organizer_id	uuid, FK -> users.id	Indexed
listing_type	enum(event, recurring, commercial)	Only 'event' used today
name	varchar	
venue_name	varchar	
latitude / longitude	decimal	
event_date	date	Applies to listing_type = event
status	enum(draft, published, ongoing, completed, cancelled)	

parking_capacity

Column	Type	Notes
id	uuid, PK	
listing_id	uuid, FK -> listings.id	Renamed from event_id
vehicle_type	enum(car, bike, other)	Unique with listing_id
total_capacity	int	
booked_count	int, default 0	Target of the atomic capacity-check update
price	decimal	

bookings

Column	Type	Notes
id	uuid, PK	
parking_capacity_id	uuid, FK	
driver_id	uuid, FK -> users.id	
vehicle_number	varchar	
quantity	int	
amount	decimal	Price snapshot at booking time
status	enum(pending, confirmed, cancelled, completed, no_show)	
idempotency_key	varchar, unique	Safe retries
qr_token	varchar, unique	Digital pass

payments

Kept separate from bookings because payment status and booking status are two different state machines — a booking can be pending while a payment is created but not yet captured.

Column	Type	Notes
id	uuid, PK	
booking_id	uuid, FK	
razorpay_order_id / razorpay_payment_id	varchar	
amount	decimal	
status	enum(created, captured, failed, refunded)	

settlements

Column	Type	Notes
id	uuid, PK	
organizer_id	uuid, FK	
period_start / period_end	date	
gross_amount / commission_amount / net_payable	decimal	
status	enum(pending, processing, paid, disputed)	

settlement_items

Per-booking breakdown inside a settlement, so the organizer gets an itemized report instead of a single lump figure — this is what Zomato/Swiggy restaurant partners commonly say they don't get, leading to reconciliation disputes.

Column	Type	Notes
id	uuid, PK	
settlement_id	uuid, FK	
booking_id	uuid, FK	
amount / commission_deducted	decimal	

Indexes to add

listings.organizer_id • parking_capacity.listing_id • bookings.parking_capacity_id • bookings.driver_id • bookings.idempotency_key (unique) • payments.booking_id • settlements.organizer_id • settlement_items.settlement_id

This is a working draft reflecting decisions made to date — expect the schema to evolve as the booking-flow and settlement logic are built out in detail.